

Section 3: Capability Broker & Token System

Context

The capability broker is Ferrite's core contribution — the single component that no other browser has. This section defines the token format, lifecycle, minting process, validation semantics, revocation mechanisms, and how the broker enforces the invariant that no privileged action occurs without authorization.

Design Philosophy

Traditional browsers give extensions and agents ambient authority — they can do anything the permission model allows, at any time, to any target. Ferrite eliminates ambient authority entirely by requiring unforgeable capability tokens for every privileged action. This follows the object-capability (ocap) security model pioneered by systems like Capsicum, WASI, and E-language.

Key principle: **A capability token is both the permission and the means of access.** You can't access a resource without the token. You can't forge a token without the broker. The broker only creates tokens after policy evaluation. Therefore, all access is policy-controlled and auditable.

Capability Token Format

```
CapabilityToken {
  token_id:      UUID v7 (time-sortable, unique)
  principal_id:  String (who is requesting)
                  Format: "ext:<extension-id>@<version>" or
                          "agent:<agent-id>" or
                          "user:<session-id>"
  cap_type:      Enum (one of the defined capability types)
  scope: {
    top_level_site: Origin (e.g., "https://example.com")
    frame_origin:   Origin (for iframe-aware scoping)
    tab_id:         u64 (browser tab identifier)
  }
  constraints: {
    methods:        Option<Vec<String>> (for network: ["GET", "POST"])
    url_allowlist:   Option<Vec<Pattern>> (glob patterns for URLs)
    element_selector: Option<String>      (CSS selector for DOM ops)
    rate_limit_rps:  Option<u32>           (max requests per second)
    max_bytes_per_hour: Option<u64>        (bandwidth cap)
```

```

    data_classes:      Option<Vec<String>> (e.g., ["public"] – not ["pii",
"credentials"])
  }
  risk_level:         Enum { Low, Medium, High }
  expires_at:         DateTime<Utc> (absolute expiry)
  audit_policy:       Enum { LogAll, LogDenials, LogNone }
  nonce:              [u8; 32] (cryptographic random, prevents replay)
  issued_at:          DateTime<Utc>
  chain_hash:         String (SHA-256 hash linking to previous audit entry)
  signature:          [u8; 64] (Ed25519 signature by broker's signing key)
}

```

Token Properties

- **Unforgeable:** Tokens are signed by the broker's Ed25519 signing key (generated per-session, stored in memory only). An attacker cannot create a valid token without compromising the broker process.
- **Scoped:** A token for `dom.read` on `https://example.com` in tab 3 cannot be used for `dom.read` on `https://evil.com` or in tab 4.
- **Time-bounded:** Every token has an `expires_at` field. Default TTL depends on risk level: low = 5 minutes, medium = 1 minute, high = single-use (expires after one exercise).
- **Non-transferable:** The `principal_id` is checked on every use. Extension A's token cannot be used by Extension B.
- **Auditable:** The `chain_hash` links the token to the audit log entry recording its creation.

Capability Types

Wave 1 (Months 2–3)

Type	What It Gates	Risk Default	Constraints
<code>dom.read</code>	Reading DOM elements, text content, attributes	Low	origin, tab, optional selector
<code>dom.write</code>	Clicking, typing, modifying elements	Medium-High	origin, tab, selector, element type affects risk (password field = high)

Type	What It Gates	Risk Default	Constraints
<code>network.fetch</code>	HTTP requests from extension/agent	Medium	methods, url_allowlist, rate_limit, max_bytes
<code>storage.read</code>	Reading cookies, localStorage, sessionStorage	Medium	origin, storage type
<code>storage.write</code>	Writing to cookies, localStorage, sessionStorage	Medium	origin, storage type, max bytes

Wave 2 (Months 4-5)

Type	What It Gates	Risk Default	Constraints
<code>clipboard.read</code>	Reading system clipboard	High	always step-up consent
<code>clipboard.write</code>	Writing to system clipboard	Medium	data size limit
<code>notify.show</code>	Displaying desktop notification	Low	rate limit (max 3/minute)
<code>download.initiate</code>	Saving file to disk	High for executables, Medium otherwise	file type, destination (sandboxed directory only), size limit

Future (Not Implemented in 8-Month Scope)

Type	What It Gates	Notes
<code>a11y.read</code>	Reading accessibility tree snapshot	Implemented in month 5 as part of accessibility module
<code>camera.access</code>	Webcam access	Future work
<code>microphone.access</code>	Microphone access	Future work
<code>geolocation.read</code>	Location data	Future work

Token Lifecycle



States

1. **REQUESTED**: `CapabilityRequest` received by broker. Not yet evaluated.
2. **EVALUATING**: Policy engine is processing. Timeout: 100ms. If exceeded, DENY.
3. **DENIED**: Policy evaluation returned deny, or consent was refused, or principal is not registered. Audit event recorded with denial reason.
4. **GRANTED / ACTIVE**: Token minted and returned to requester. Token can be exercised until expiry or revocation.
5. **EXPIRED**: TTL elapsed. Token is no longer valid. Any exercise attempt is denied.
6. **USED**: For single-use tokens (high-risk actions). Exercised once, then invalidated.

7. **REVOKED:** Manually revoked by user (via UI), by policy change, or by broker (e.g., tab closed, origin navigated away).

Revocation Triggers

- **User clicks "Revoke" in capability inspector panel**
 - **Tab is closed** → all tokens scoped to that tab are revoked
 - **Navigation to different origin** → all tokens scoped to the previous origin are revoked
 - **Extension is unloaded** → all tokens for that extension's principal are revoked
 - **Agent disconnects** → all tokens for that agent's principal are revoked
 - **Policy file is reloaded** → tokens granted under the old policy are re-evaluated; incompatible ones are revoked
-

Broker Interface (Messages)

CapabilityRequest (Untrusted → Broker)

```
CapabilityRequest {
  request_id:    UUID v7
  principal_id:  String
  cap_type:      CapabilityType
  scope:         Scope
  action_detail: ActionDetail (context-specific: selector, URL, etc.)
  justification: Option<String> (human-readable reason, for audit)
}
```

CapabilityResponse (Broker → Untrusted)

```
CapabilityResponse {
  request_id:    UUID v7 (matches request)
  outcome:       Enum { Granted(CapabilityToken), Denied(DenialReason) }
  audit_ref:     String (hash of the audit event for this decision)
}

DenialReason {
  code:          Enum { PolicyDeny, ConsentRefused, PrincipalUnknown,
                        CapabilityNotInManifest, RateLimited, Expired,
                        ScopeViolation, BrokerOverloaded, InternalError }
  rule_name:     Option<String> (which Rego rule caused denial)
```

```
message:      String (human-readable)
}
```

ConsentRequest (Broker → UI)

```
ConsentRequest {
  request_id:      UUID v7
  principal_id:    String
  cap_type:        CapabilityType
  scope:           Scope
  action_detail:   ActionDetail
  risk_level:      RiskLevel
  risk_reason:     String (why this was classified as high risk)
}
```

ConsentResponse (UI → Broker)

```
ConsentResponse {
  request_id:      UUID v7
  decision:        Enum { Allow, Deny, AllowOnce, AllowForSession }
  user_note:       Option<String> (optional user annotation)
}
```

Broker Internal Design

Token Storage

Active tokens are stored in a `HashMap<TokenId, CapabilityToken>` in the broker's memory. This map is never persisted to disk (tokens are session-only). The map is indexed by:

- `token_id` → O(1) lookup for validation
- `principal_id` → all tokens for a principal (for bulk revocation)
- `(tab_id, origin)` → all tokens scoped to a tab+origin (for navigation revocation)

Validation Fast Path

Token validation must be <1ms. The fast path:

1. Look up token by ID in HashMap → O(1)
2. Check `expires_at > now` → single comparison

3. Check `principal_id` matches requester → string equality
4. Check `scope` matches current context → struct equality
5. Verify `signature` → Ed25519 verify (< 100μs)
6. If all pass → VALID. Emit audit event.
7. If any fail → INVALID. Emit audit event with failure reason.

No policy re-evaluation on the fast path. Policy was evaluated at minting time. The token encodes the result.

Signing Key Management

- Ed25519 keypair generated at broker startup using OS CSPRNG
 - Private key held in memory only — never written to disk
 - Public key available to all components for verification
 - Key rotation: new keypair on every browser restart
 - Future work: HSM/TPM-backed keys for stronger guarantees
-

Loopholes Analyzed and Mitigated

Loophole 1: Token replay across sessions

Attack: Attacker captures a valid token and replays it in a later session. **Mitigation:** Tokens are signed with a per-session keypair. A token from session A is invalid in session B because the signing key is different. Additionally, the nonce prevents replay within a session.

Loophole 2: Token theft by malicious extension

Attack: Extension A steals Extension B's token from shared memory. **Mitigation:** Tokens are not stored in shared memory. Each extension's host function receives tokens via the broker's response channel, which is per-extension. The Wasm sandbox prevents cross-extension memory access.

Loophole 3: Time-of-check-to-time-of-use (TOCTOU)

Attack: Token is valid at check time but the context changes before use (e.g., page navigates away). **Mitigation:** The engine verifies the token with the broker immediately before executing the action (second check). If the context has changed (different origin, tab closed), the token is invalid. The broker maintains current tab/origin state.

Loophole 4: Broker denial-of-service

Attack: Malicious extension floods the broker with capability requests to slow it down.

Mitigation: Per-principal rate limiting on the broker's input channel. If a principal exceeds 100 requests/second, subsequent requests are auto-denied for 1 second. This is enforced before policy evaluation.

Loophole 5: Consent dialog fatigue

Attack: Agent repeatedly triggers high-risk actions hoping user will eventually click "Allow" out of fatigue. **Mitigation:** If the same principal triggers 3 consecutive consent denials, the principal is suspended for 5 minutes. The UI shows a clear warning. "AllowForSession" option lets the user make a conscious one-time decision to trust the agent for the session.

Loophole 6: Scope escalation via frame injection

Attack: Agent navigates a tab to a trusted origin, gets a token, then the page is redirected to an untrusted origin. The token's scope still says the trusted origin. **Mitigation:** Tokens are scoped to `(tab_id, top_level_site)`. On navigation, the broker is notified and revokes all tokens for the previous origin on that tab. The Servo/CEF integration layer fires a navigation event that triggers revocation.

Loophole 7: Policy engine bypass via malformed input

Attack: Attacker crafts a capability request with unusual characters or structure that causes the Rego policy to error out, hoping the error path grants access. **Mitigation:** Cross-cutting invariant: all errors default to DENY. If Regorus returns an error, evaluation result, or panic, the broker treats it as a denial. Input sanitization ensures all strings are valid UTF-8 with length limits before policy evaluation.